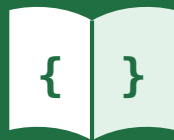

Module 16 — Async, Scheduling and Spring Batch

Lesson Notes

Spring Boot Edu



2026-09-24

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Three Kinds of “Not Now”	2
2.2 Chunk-Oriented Processing	2
2.3 Job, Job Instance, Job Execution	3
3. Step-by-Step Examples	3
3.1 @Async and CompletableFuture	3
3.2 @Scheduled	5
3.3 A Spring Batch Job: Reader, Processor, Writer	6
3.4 The Step: Chunks, Skip and Retry	8
3.5 Job Parameters and Restart	9
3.6 Testing	10
4. Common Mistakes and Best Practices	10
5. Summary	10
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Run slow calls in parallel with `@Async` and `CompletableFuture`
- Let `@Async` and `@Scheduled` run on virtual threads
- Schedule work with `@Scheduled` (fixed delay and cron) and know what changes with several instances
- Build a Spring Batch 6 job: reader, processor, writer, chunks
- Make a job robust: skip bad data, retry temporary failures, restart after a crash
- Test async code, schedules and batch jobs

Prerequisites: Module 06 (PostgreSQL, transactions) · **Estimated time:** 5 hours · **Docker required**

2. Concepts

2.1 Three Kinds of “Not Now”

Tool	Runs	Typical use
<code>@Async</code>	now, but on another thread; the caller does not wait	call three slow services at once
<code>@Scheduled</code>	later, at a fixed rate or at fixed times	clean up every 30 s, report at 02:00
Spring Batch	a large amount of data, in chunks, restartable	import 2 million CSV lines every night

2.2 Chunk-Oriented Processing

A batch step reads, processes and writes items in **chunks**. Each chunk is one transaction:

```
read ×10 → process ×10 → write 10 → COMMIT    (chunk 1: items 1–10)
read ×10 → process ×10 → write 10 → COMMIT    (chunk 2: items 11–20)
...
```

After every commit, Spring Batch saves in its **job repository** (database tables) how far the step got. That is what makes a restart possible: a failed job continues after the last committed chunk.

2.3 Job, Job Instance, Job Execution

Term	Meaning	Example
Job	the definition	importBooksJob
Job instance	job + identifying parameters	importBooksJob for input.file=books-2026-09.csv
Job execution	one attempt to run an instance	the run on Monday that failed, the restart on Tuesday

A job instance that has completed cannot run again. A failed one can be restarted as many times as needed.

3. Step-by-Step Examples

With Docker running, start the application. PostgreSQL starts from the root compose.yaml:

```
./mvnw -pl modules/16-async-scheduling-batch/lesson spring-boot:run
```

The tour output:

```
== 3.1 @Async on virtual threads
5 price calls of 200 ms: total 424.90 in 202 ms
== 3.3 A Spring Batch job
status COMPLETED: read 7, written 5, skipped 2
== 3.5 The same parameters again
refused: A job instance already exists and is complete for identifying parameters=...
```

3.1 @Async and CompletableFuture

```
@Service
public class PriceClient {

    private static final Map<String, BigDecimal> PRICES = Map.of(
        "9780134685991", new BigDecimal("89.90"), "9781617297571", new
        ↪ BigDecimal("95.00"),
```

```

        "9780321336781", new BigDecimal("55.00"), "9781449373320", new
        ↪ BigDecimal("110.00"),
        "9781492078005", new BigDecimal("75.00"));

private final Set<String> threadsSeen = ConcurrentHashMap.newKeySet();

@Async // runs on the task
↪ executor, not on the caller
public CompletableFuture<BigDecimal> priceOf(String isbn) {
    Thread current = Thread.currentThread();
    threadsSeen.add((current.isVirtual() ? "virtual:" : "platform:") +
    ↪ current.getName());
    sleep(200); // the remote call
    BigDecimal price = PRICES.get(isbn);
    return price == null
        ? CompletableFuture.failedFuture(new NoSuchElementException("No price for
        ↪ " + isbn))
        : CompletableFuture.completedFuture(price);
}

```

```

@Service
public class PriceAggregator {

    private final PriceClient prices;

    public PriceAggregator(PriceClient prices) {
        this.prices = prices; // another bean: calls
        ↪ go through the proxy
    }

    public BigDecimal totalFor(List<String> isbnns) {
        List<CompletableFuture<BigDecimal>> calls =
        ↪ isbnns.stream().map(prices::priceOf).toList(); // all started
        return calls.stream()
            .map(CompletableFuture::join) // now wait
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}

```

- `@EnableAsync` switches the feature on. An `@Async` method runs on the task executor, and the caller gets a `CompletableFuture` at once.
- `PriceAggregator` first **starts** all calls, then **waits** for all of them: five calls of 200 ms take about 200 ms.
- With `spring.threads.virtual.enabled=true`, Boot's executor starts one virtual thread per task. `PriceAggregatorTest` checks that the calls ran on virtual threads.

! Important

@Async works through a proxy, like @Transactional. A call from a method **in the same class** runs synchronously, on the caller's thread. That is why PriceAggregator and PriceClient are two beans.

★ Tip

An exception inside an @Async method completes the future exceptionally, so the caller sees it with join() or exceptionally(...). For @Async methods that return void, nobody sees it, except an AsyncUncaughtExceptionHandler.

3.2 @Scheduled

The intervals come from the configuration:

```
bookstore:
  snapshot:
    interval: 30s                                # @Scheduled(fixedDelayString = ...)
  report:
    cron: "0 0 2 * * *"                          # every night at 02:00 (second minute hour
↪    day month weekday)
```

```
@Component
public class StockSnapshotTask {

    private final AtomicInteger runs = new AtomicInteger();

    @Scheduled(fixedDelayString = "${bookstore.snapshot.interval}") // 30s, 100ms, ...
    ↪    from application.yaml
    void takeSnapshot() {
        runs.incrementAndGet();                                // a real task
    ↪    would copy the stock levels
    }

    public int runs() {
        return runs.get();
    }
}
```

```
@Component
class NightlyReportTask {

    private static final Logger log = LoggerFactory.getLogger(NightlyReportTask.class);
```

```

@Scheduled(cron = "${bookstore.report.cron}", zone = "Europe/Istanbul")
void createReport() {
    log.info("nightly report created");           // with several
↪ instances: see ShedLock (3.2)
}

```

Attribute	Means
fixedDelay	wait this long after the previous run has finished
fixedRate	start every period, even if the previous run is still busy
cron	at these times: second minute hour day month weekday, e.g. 0 0 2 * * * = every day at 02:00

- Always set zone for cron jobs: servers and containers often run in UTC.
- A cron expression is tested by asking for the next run (`CronExpression.parse(...).next(...)`), not by waiting until 2 am (`NightlyReportCronTest`).

! Warning

`@Scheduled` runs in **every** instance of the application. With three instances, the nightly report is created three times. The usual remedy is a distributed lock: **ShedLock** stores a lock row in the database (or Redis, ...), and only the instance that gets the lock runs the task. Alternatively, move such work to a single scheduler: a Kubernetes `CronJob` (module 22) or a Batch job started once.

3.3 A Spring Batch Job: Reader, Processor, Writer

The job repository lives in PostgreSQL. Boot creates its tables and does not start jobs by itself (under `spring`):

```

batch:
  jdbc:
    initialize-schema: always           # create BATCH_JOB_INSTANCE,
↪ BATCH_STEP_EXECUTION, ... in PostgreSQL
  job:
    enabled: false                     # do not run jobs at startup; LessonTour
↪ starts them with parameters

```

The **reader** reads one CSV line after the other. It is step-scoped, so it gets the file from the job parameters:

```
@Bean
@StepScope // a new reader per step run,
↪ with the job's parameters
FlatFileItemReader<BookLine> bookLineReader(@Value("#{jobParameters['input.file']}")
↪ Resource input) {
    return new FlatFileItemReaderBuilder<BookLine>()
        .name("bookLineReader") // the key of its position in
        ↪ the ExecutionContext
        .resource(input)
        .linesToSkip(1) // the header line
        .delimited().names("isbn", "title", "price")
        .targetType(BookLine.class) // records are supported
        .build();
}
```

The **processor** validates and converts a line. It is plain Java, so BookLineProcessorTest tests it without a job:

```
@Component
public class BookLineProcessor implements ItemProcessor<BookLine, ImportedBook> {

    private final ImportFaults faults;

    public BookLineProcessor(ImportFaults faults) {
        this.faults = faults;
    }

    @Override
    public ImportedBook process(BookLine line) {
        faults.check(line.isbn());
        if (!line.isbn().matches("\\d{13}")) {
            throw new InvalidBookLineException("invalid ISBN: " + line.isbn());
        }
        BigDecimal price = parsePrice(line.price());
        if (price.signum() < 0) {
            throw new InvalidBookLineException("negative price for " + line.isbn());
        }
        return new ImportedBook(line.isbn(), line.title().trim(), price);
    }
}
```

The **writer** writes a whole chunk in one JDBC batch. ON CONFLICT ... DO UPDATE makes it idempotent, so writing the same book twice (e.g. after a restart) does no harm:

```

@Bean
JdbcBatchItemWriter<ImportedBook> importedBookWriter(dataSource) {
    return new JdbcBatchItemWriterBuilder<ImportedBook>()
        .dataSource(dataSource)
        .sql("""
            INSERT INTO imported_book (isbn, title, price) VALUES (:isbn, :title,
↪ :price)
            ON CONFLICT (isbn) DO UPDATE SET title = EXCLUDED.title, price =
↪ EXCLUDED.price,
                                                    imported_at = now()""") //
↪ idempotent: safe to rerun
        .itemSqlParameterSourceProvider(book -> new MapSqlParameterSource()
            .addValue("isbn", book.isbn()).addValue("title",
↪ book.title()).addValue("price", book.price()))
        .build();
}

```

3.4 The Step: Chunks, Skip and Retry

```

@Bean
Step importStep(JobRepository jobRepository, PlatformTransactionManager
↪ transactionManager,
                FlatFileItemReader<BookLine> reader, BookLineProcessor processor,
                JdbcBatchItemWriter<ImportedBook> writer, ImportSkipLog skipLog) {
    return new StepBuilder("importStep", jobRepository)
        .<BookLine, ImportedBook>chunk(10) // 10 items per
        ↪ transaction
        .transactionManager(transactionManager)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .faultTolerant()
        .skip(InvalidBookLineException.class) // bad data: skip
        ↪ the item ...
        .skip(FlatFileParseException.class)
        .skipLimit(10) // ... but not more
        ↪ than 10 times
        .retry(TransientDataAccessException.class) // a short
        ↪ database hiccup: try again
        .retryLimit(3)
        .skipListener(skipLog)
        .build();
}

@Bean

```

```
Job importBooksJob(JobRepository jobRepository, Step importStep) {
    return new JobBuilder("importBooksJob", jobRepository)
        .start(importStep)
        .build();
}
```

- `chunk(10)`: 10 items per transaction. Bigger chunks are faster, smaller chunks lose less work on a failure.
- **Skip** is for bad data that will never become good: the broken ISBN and the negative price of the sample file are skipped and logged (`ImportSkipLog`). More than 10 skips fail the job: too much bad data usually means a wrong file.
- **Retry** is for temporary failures, such as a database that is briefly unavailable. The same item is tried again, up to 3 times.
- In Spring Batch 6, `chunk(int)` returns the new `ChunkOrientedStepBuilder`. Its retry support uses the retry API of Spring Framework 7.

Look at the job repository after a run:

```
docker compose exec postgres psql -U bookstore -c \
"SELECT step_name, read_count, write_count, process_skip_count, commit_count, status
  FROM batching.batch_step_execution"
```

step_name	read_count	write_count	process_skip_count	commit_count	status
importStep	7	5	2	1	COMPLETED

3.5 Job Parameters and Restart

The job is started with parameters. `input.file` identifies the job instance:

```
var parameters = new JobParametersBuilder()
    .addString("input.file", "classpath:import/books-2026-09.csv")
    .toJobParameters();
JobExecution execution = jobOperator.start(importBooksJob, parameters);
```

`ImportBooksJobTest` shows what the job repository makes possible:

1. **Completed instances do not run twice.** Starting the same parameters again throws `JobInstanceAlreadyCompleteException`. A nightly import therefore adds the date to its parameters.
2. **Failed instances restart where they stopped.** The test imports 30 lines and simulates a crash at line 25. Chunks 1 and 2 are committed (20 rows), and the job is `FAILED`. After the problem is fixed, `jobOperator.restart(execution)` reads only lines 21–30, and the job ends `COMPLETED` with all 30 rows.

3.6 Testing

- `@Async`: measure that parallel calls take about as long as one, and check the thread (`Thread.isVirtual()`).
- `@Scheduled`: make the interval short with a property (`bookstore.snapshot.interval=100ms`) and wait with `Awaitility` until the task has run.
- Spring Batch: `@SpringBatchTest` provides `JobOperatorTestUtils` (start jobs and single steps) and `JobRepositoryTestUtils` (remove old runs, so that tests are independent).

4. Common Mistakes and Best Practices

!! Caution

A batch writer must be idempotent. After a crash, the last chunk is processed again. With a plain `INSERT`, the restart fails on duplicate keys or creates duplicates.

- **Do:** start all `@Async` calls before waiting for the first result. `priceOf(a).join(); priceOf(b).join();` is sequential again.
- **Don't:** call an `@Async` method from the same class. The call bypasses the proxy and runs synchronously.
- **Do:** give cron schedules an explicit time zone, and protect them against several instances (`ShedLock` or a single scheduler).
- **Don't:** skip exceptions that mean “try again later”, or retry exceptions that mean “never”. Skip bad data, retry temporary failures.
- **Do:** put everything that identifies a run into the job parameters (file, date), so that reruns and restarts behave predictably.
- **Don't:** load a whole file into memory. Readers stream the data chunk by chunk.

5. Summary

- `@Async` returns a `CompletableFuture` at once. Start all calls, then wait: the waits overlap. With virtual threads, blocking in the task is cheap.
- `@Scheduled` runs with a fixed delay, a fixed rate or a cron expression. With several instances, a lock such as `ShedLock` is needed.
- A Spring Batch step reads, processes and writes in chunks. Each chunk is one transaction.
- `Skip` is for bad data, `retry` for temporary failures. The job repository records the progress.
- Job parameters identify a job instance. Completed instances do not run again, and failed ones restart after the last committed chunk.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — Task Execution and Scheduling](#)
- [Spring Boot — Task Execution and Scheduling](#)
- [Spring Batch Reference · Chunk-oriented Processing](#)
- [Spring Boot — Batch Applications](#)
- [ShedLock](#)